

Benutzerhandbuch

Vanessa Künzel & Jasmin Hundt

Link zum Repository: <https://github.com/jasmin261098/lernzeit-manager>

Fachliche Dokumentation

Der Lernzeit-Manager ist eine Webanwendung für Studierende, die bei der Planung, Nachverfolgung und Einhaltung von Lernzeiten und Lernzielen unterstützt. Das System hilft dabei, Lernzeiten gemäß dem vorgesehenen Workload aus Modulhandbüchern einzuplanen, die Einhaltung zu überprüfen und die Planung bei Bedarf anzupassen.

Fachliche Prozesse

Prozess: Registrierung und Anmeldung

Neuer Nutzer → Registrierungsformular (E-Mail + Passwort)

- Validierung (E-Mail-Format, min. 8 Zeichen Passwort)
- Passwort-Hash wird erstellt (bcrypt, 12 Runden)
- Benutzerkonto wird angelegt
- Automatische Anmeldung nach Registrierung
- JWT-Token (Gültigkeit 7 Tage) wird ausgestellt
- Weiterleitung zum Dashboard

Prozess: Lernziel anlegen und verfolgen

Nutzer → Neue Lernziel anlegen (Titel, Stundenziel, Zeitraum)

- Ziel wird mit Status "offen" gespeichert
- Nutzer startet Timer-Sessions, optional mit Zielzuordnung
- Geleistete Stunden werden automatisch summiert
- Dashboard zeigt Fortschrittsbalken (0–100 %)
- Nutzer setzt Status manuell auf "in Bearbeitung" / "abgeschlossen"

Prozess: Lernzeit mit dem Fokus-Timer erfassen

Nutzer → Thema und optionales Lernziel eingeben → Timer starten

- Session wird in der Datenbank mit Startzeit angelegt
- Timer kann pausiert und fortgesetzt werden (client-seitig)
- Timer stoppen → Endzeit und Dauer (Minuten) werden gespeichert
- Session erscheint in der Verlaufsanzeige

Wichtig: Es kann immer nur eine aktive Session pro Nutzer gleichzeitig laufen. Beim Schließen des Browser-Tabs wird der Nutzer gewarnt, wenn eine Session noch aktiv ist.

Prozess: Lernzeitplanung

Nutzer → Neuen Lernplan erstellen (Titel, Zeitraum)

- Optionale Monatspläne mit geplantem Stundenumfang hinterlegen
- Monatsübersicht: Pläne nach Monat gefiltert anzeigen
- Pläne und Monateinträge können jederzeit bearbeitet oder gelöscht werden

Prozess: Erinnerungen und Benachrichtigungen

Nutzer → Erinnerung erstellen (Nachricht + Zeitpunkt)

- Backend-Cronjob prüft jede Minute fällige Erinnerungen
- Fällige Erinnerungen werden als "gesendet" markiert
- In-App-Benachrichtigungsglocke zeigt ungelesene Erinnerungen
- Inaktivitäts-Warnung: Wenn letzte Session > 3 Tage zurückliegt, erscheint ein Hinweis in der Benachrichtigung Glocke

Fachliche Konzepte

Lernziel (LearningGoal)

Ein Lernziel beschreibt ein konkretes Vorhaben eines Nutzers (z. B. „Modul Mathematik abschließen“) mit einem definierten Stundenziel und einem Zeitraum. Es durchläuft die Statusübergänge: offen → in Bearbeitung → abgeschlossen. Dem Ziel können einzelne Lern-Sessions zugeordnet werden, um den Fortschritt automatisch zu berechnen.

Lernplan (LearningPlan) & Monatsplan (MonthlyPlan)

Ein Lernplan ist eine Grobplanung über einen Zeitraum (z.B. 6 Monate). Innerhalb eines Lernplans können Monatspläne angelegt werden, die für einen spezifischen Monat die geplanten Stunden und Notizen festhalten. Monatspläne werden beim Löschen des übergeordneten Lernplans automatisch mitgelöscht (Cascade Delete).

Lernsession (StudySession)

Eine Lernsession ist eine einzelne gemessene Lerneinheit mit Startzeit, Endzeit und Dauer in Minuten. Sessions können einem Lernziel zugeordnet werden. Eine Session gilt als offen, solange keine Endzeit gesetzt ist.

Erinnerung (Reminder)

Erinnerungen sind Hinweise mit Nachricht und Zeitpunkt. Ein serverseitiger Cronjob prüft jede Minute, ob Erinnerungen fällig sind, und markiert sie als gesendet.

Dashboard

Das Dashboard aggregiert alle Lern-Sessions eines Nutzers und berechnet: Gesamtstunden (alle Sessions), Durchschnittliche Zielerreichung (Prozent), Anzahl abgeschlossener Ziele sowie den Fortschritt je Lernziel (geloggte Stunden vs. Zielstunden, maximal 100 %).

Geschäftsregeln

- Eine E-Mail-Adresse kann nur einmal als Benutzerkonto registriert werden.
- Das Passwort muss bei der Registrierung mindestens 8 Zeichen lang sein.

- Es kann zu einem Zeitpunkt pro Nutzer maximal eine aktive Timer-Session existieren.
- Der Fortschritt eines Lernziels wird ausschließlich aus Sessions berechnet, die explizit diesem Ziel zugeordnet wurden.
- Der angezeigte Fortschrittswert eines Lernziels ist auf 100 % begrenzt, auch wenn die geloggte Zeit das Stundenziel überschreitet.
- Alle CRUD-Operationen auf Lernziele, Pläne, Sessions und Erinnerungen sind auf den jeweils authentifizierten Benutzer beschränkt (keine Fremdressourcen zugänglich).
- Lernziele können folgende Status-Werte haben: open (offen), in_progress (in Bearbeitung), done (abgeschlossen). Andere Werte sind ungültig.
- Beim Löschen eines Lernplans werden alle zugehörigen Monatspläne automatisch mitgelöscht.
- Die Inaktivitäts-Warnung wird ausgelöst, wenn die letzte abgeschlossene Session mehr als 3 Tage zurückliegt.
- JWT-Token sind 7 Tage gültig. Nach Ablauf muss sich der Nutzer erneut anmelden.

Technische Dokumentation

Systemarchitektur

Der Lernzeit-Manager folgt einer klassischen Client-Server-Architektur mit zwei eigenständigen Diensten, die über Docker Compose orchestriert werden.

Browser

React SPA (Vite, Port 5173)

HTTP /api/* (Vite-Proxy → http://backend:3000)



Backend

Express.js REST-API (Node.js, Port 3000)

Prisma ORM

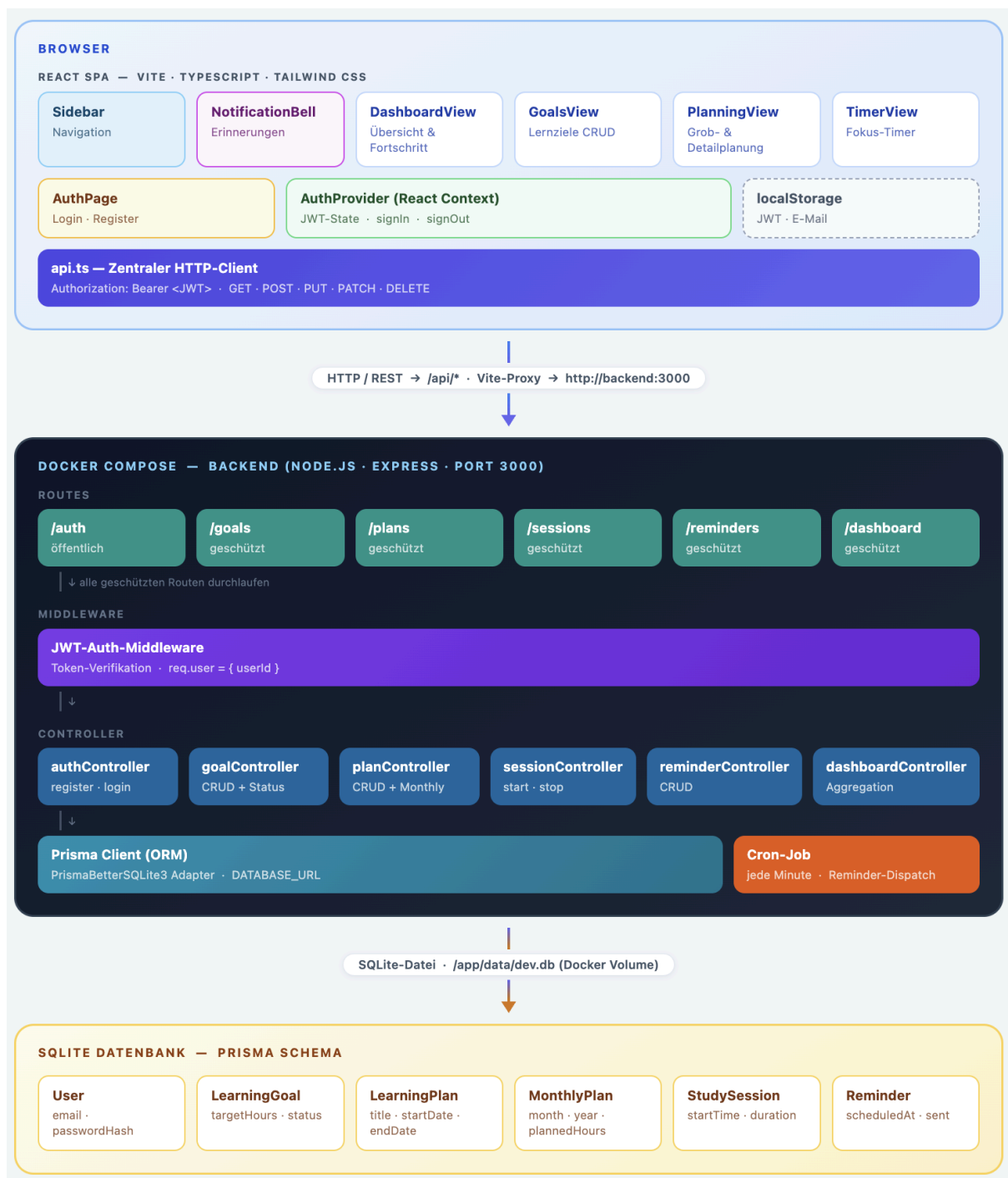


Datenbank

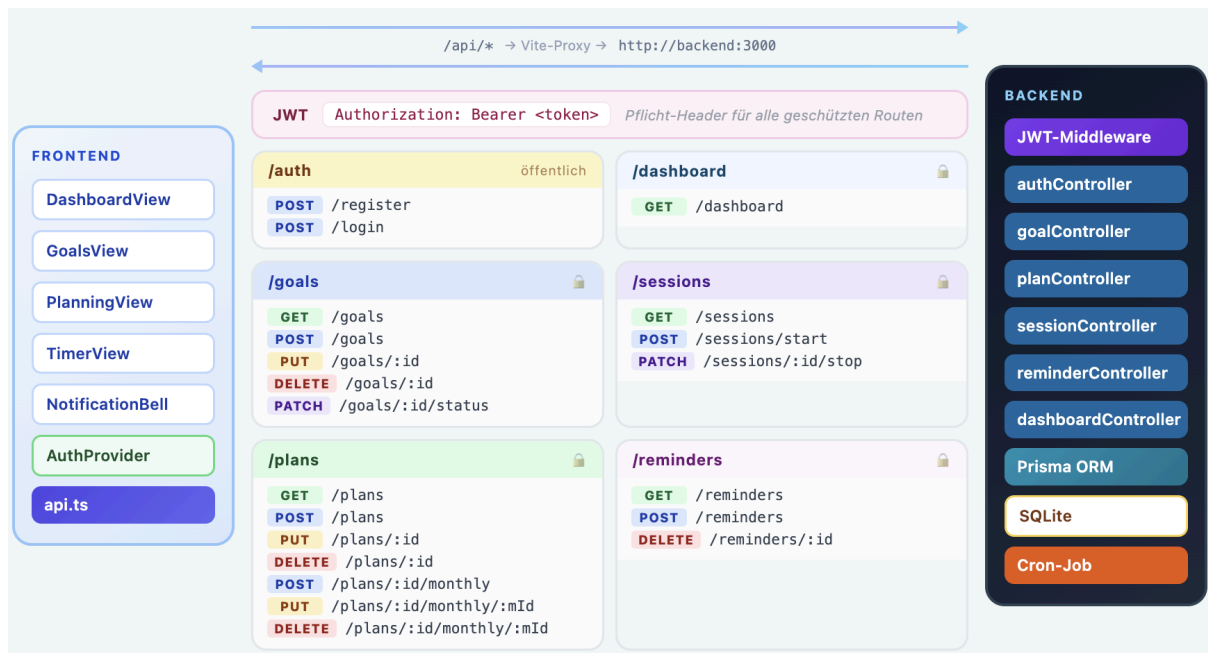
SQLite (Datei: /app/data/dev.db, Docker Volume)

Das Frontend kommuniziert ausschließlich über das Vite-Proxy-Setup mit dem Backend. Alle Anfragen des Browsers an /api/* werden proxy-seitig an http://backend:3000 weitergeleitet.

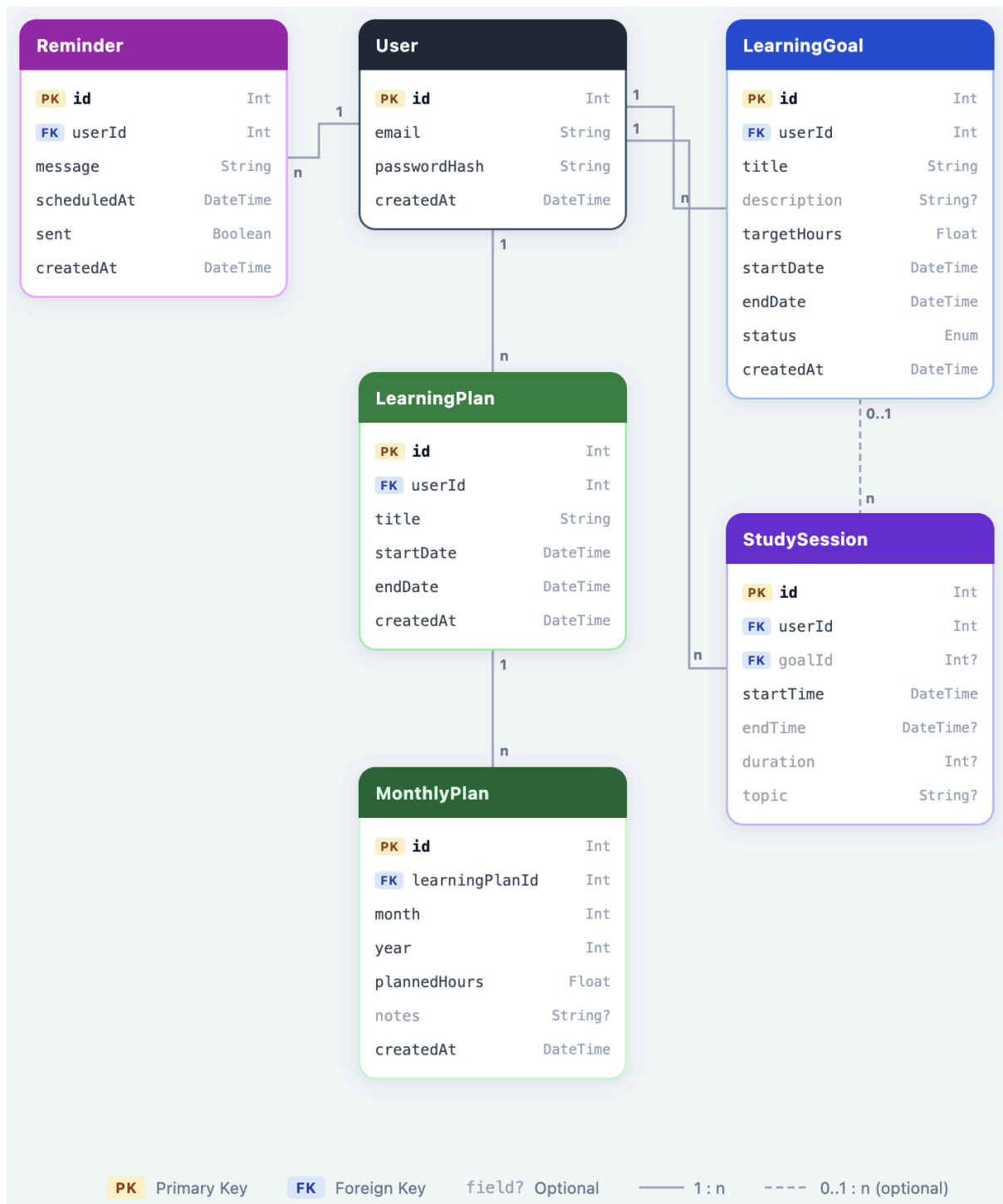
Komponentendarstellung



Schnittstellendarstellung



Datenbankdiagramm



Betriebsdokumentation

Voraussetzungen

- Docker (Version 24.x) als Engine
- Docker Compose (Version 2.x) zur Orchestrierung
- Git zum Klonen des Repositories

Kein lokales Node.js erforderlich – alle Abhängigkeiten werden innerhalb der Docker-Container aufgelöst.

Installation und Start

1. Repository klonen

<https://github.com/jasmin261098/lernzeit-manager>

```
cd lernzeit-manager
```

2. Umgebungsvariablen konfigurieren

```
cp backend/.env.example backend/.env
```

Mindestens JWT_SECRET in backend/.env auf einen sicheren, zufälligen Wert setzen (siehe Abschnitt 3.3)

3. Anwendung starten

```
docker compose up --build
```

Der erste Start kann mehrere Minuten dauern, da Node.js-Abhängigkeiten installiert werden. Das Backend führt beim Start automatisch folgende Schritte aus (entrypoint.sh):

- Datenbankverzeichnis/app/data erstellen
- `prisma generate` – Prisma-Client generieren
- `prisma db push` – Datenbankschema anlegen/aktualisieren
- `npm run dev` – Entwicklungsserver starten

4. Anwendung aufrufen

Frontend (Browser): `http://localhost:5173`

Backend-API (direkt): `http://localhost:3000`

Anwendung stoppen: `docker compose down`

Zum Stoppen mit Datenlöschung (Volume entfernen): `docker compose down -v`

Konfiguration

Die Konfiguration des Backends erfolgt über backend/.env:

Variable	Standardwert	Beschreibung
DATABASE_URL	file:/app/data/dev.db	Pfad zur SQLite-Datenbankdatei im Container
JWT_SECRET	change-this-to-a-random-secret-before-deploying	Geheimschlüssel für JWT-Signierung – muss vor dem ersten Start geändert werden
PORT	3000	Port des Backend-Servers

Empfohlene Generierung eines sicheren JWT-Secrets: `openssl rand -base64 48`

Das Frontend benötigt keine separate .env-Datei. Der Proxy-Target (`http://backend:3000`) ist in frontend/lernzeit-manager/vite.config.ts fest konfiguriert und funktioniert im Docker-Netzwerk.

Demo-Account

Beim ersten Start legt die Anwendung automatisch einen Demo-Account mit Beispieldaten an:

E-Mail: demo@lernzeit.de

Passwort: demo1234

Der Demo-Account enthält folgende Beispieldaten:

- 3 Lernziele (z. B. „TypeScript lernen“, „React vertiefen“)
- 1 Lernplan mit 6 Monatsplänen (Januar–Juni)
- 5 abgeschlossene Lern Sessions
- 2 Erinnerungen

Der Demo-Account dient nur als Beispiel, der Benutzer kann sich über den Button Registrierung einen eigenen Account anlegen.

Testabschlussbericht

Frontend Test Zusammenfassung

{...}

Frontend Testfälle

{...}

Backend Test Zusammenfassung

Teststart	Unit-Tests (Backend)
Test-Framework	Vitest 4.x
Testdatum	05.07.2026
Anzahl Testdateien	5
Anzahl Testfälle gesamt	32
Testfälle bestanden	32
Testfälle fehlgeschlagen	0
Testabdeckung (getestete Controller)	Auth-Middleware, Auth-Controller, Goal-Controller, Session-Controller, Dashboard-Controller
Gesamtdauer der Testausführung	256 ms
Testergebnis	BESTANDEN 

Alle Tests verwenden gemockte Abhängigkeiten (Prisma-Client, bcrypt, jsonwebtoken), um Unit-Tests isoliert vom Dateisystem und der Datenbank ausführen zu können. Die Testisolation wird durch `vi.clearAllMocks()` in `beforeEach`-Hooks sichergestellt.

Backend Testfälle

Testmodul 1: Auth-Middleware (`auth.middleware.test.js`)

- Fehlender Authorization-Header ✓
- Header mit falschem Schema (nicht „Bearer“) ✓
- Ungültiger/abgelaufener JWT-Token ✓
- Gültiger Bearer-Token ✓

Testmodul 2: Auth-Controller – Registrierung (`authController.test.js`)

- Registrierung ohne E-Mail ✓
- Registrierung ohne Passwort ✓
- Erfolgreiche Registrierung ✓
- Registrierung mit bereits vergebener E-Mail ✓

Testmodul 3: Auth-Controller – Login (`authController.test.js`)

- Login mit unbekannter E-Mail ✓
- Login mit falschem Passwort ✓
- Erfolgreicher Login ✓

Testmodul 4: Lernziel-Controller (`goalController.test.js`)

- Alle Ziele des Nutzers abrufen ✓
- Neues Ziel anlegen ✓
- Ziel aktualisieren – nicht gefunden ✓
- Ziel erfolgreich aktualisieren ✓
- Ziel löschen – nicht gefunden ✓
- Ziel erfolgreich löschen ✓
- Status-Update mit ungültigem Wert ✓
- Status-Update auf open ✓
- Status-Update auf in_progress ✓
- Status-Update auf done ✓

Testmodul 5: Session-Controller (`sessionController.test.js`)

- Alle Sessions abrufen ✓
- Session starten – bereits aktive Session vorhanden ✓
- Session erfolgreich starten ✓
- Session stoppen – Session nicht gefunden ✓
- Session stoppen – Session bereits beendet ✓
- Session erfolgreich stoppen ✓

Testmodul 6: Dashboard-Controller (`dashboardController.test.js`)

- Dashboard ohne Sessions und Ziele ✓
- Gesamtstunden korrekt berechnen ($60 + 30 \text{ min} = 1,5 \text{ h}$) ✓
- Zielfortschritt: `loggedHours` und `percent` berechnen ($300 \text{ min} / 10 \text{ h} = 50 \%$) ✓
- Fortschritts-Prozentwert auf 100 % begrenzen ($240 \text{ min} / 2 \text{ h} = 200 \% \rightarrow 100 \%$) ✓
- Sessions eines Ziels nicht auf anderes Ziel anrechnen ✓